

BAB 7

METODE EVALUASI KLASIFIKASI

7.1 TUJUAN PERCOBAAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Mengetahui prinsip kerja metode evaluasi klasifikasi
2. Mengimplementasikan berbagai macam metode evaluasi pada kasus klasifikasi

7.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab
- Pustaka: pandas, numpy, matplotlib, seaborn

7.3 DASAR TEORI

Ada beberapa metrik penilaian untuk melakukan evaluasi terhadap metode klasifikasi di dalam pembelajaran mesin. Metrik ini berguna untuk mengevaluasi kinerja model klasifikasi dan memilih model terbaik berdasarkan kebutuhan spesifik. Dengan memahami dan menggunakan metrik ini, kita dapat memperoleh informasi tentang bagaimana model kita bekerja dan bagaimana meningkatkan performanya. Berikut adalah beberapa metode yang bisa digunakan:

7.3.1 CONFUSION MATRIX

Confusion Matrix adalah alat evaluasi yang menunjukkan kinerja dari model klasifikasi pada sekumpulan data uji dengan format tabel $N \times N$, di mana N adalah jumlah kelas. Matriks ini terdiri dari 4 komponen utama:

- True Positive (TP): Jumlah sampel yang diklasifikasikan sebagai kelas positif secara benar.
- True Negative (TN): Jumlah sampel yang diklasifikasikan sebagai kelas negatif secara benar.
- False Positive (FP): Jumlah sampel yang salah diklasifikasikan sebagai kelas positif.
- False Negative (FN): Jumlah sampel yang salah diklasifikasikan sebagai kelas negatif.

Pada kasus jumlah kelas lebih dari satu, penentuan TP, TN, FP, dan FN dilakukan seperti contoh berikut:

		Prediksi		
		Kelas 1	Kelas 2	Kelas 3
Ground Truth	Kelas 1	100	20	10
	Kelas 2	15	80	5
	Kelas 3	10	5	90

Maka penentuan keempat metrik tersebut adalah:

Untuk Kelas 1:
TP = 100
FP = 20 + 10 = 30
FN = 15 + 10 = 25
TN = 80 + 90 = 170

Untuk Kelas 2:
TP = 80
FP = 15 + 5 = 20
FN = 20 + 5 = 25
TN = 100 + 90 = 190

Untuk Kelas 3:
TP = 90
FP = 10 + 5 = 15
FN = 10 + 5 = 15
TN = 100 + 80 = 180

7.3.2 PRECISION

Precision atau presisi mengukur ketepatan dari prediksi positif model, yaitu berapa banyak dari prediksi yang diklasifikasikan sebagai positif adalah benar-benar positif.

$$Precision = \frac{TP}{TP + FP}$$

7.3.3 RECALL

Recall atau sensitivitas adalah rasio dari jumlah sampel positif yang benar-benar terdeteksi oleh model dibandingkan dengan total sampel positif yang ada.

$$Recall = \frac{TP}{TP + FN}$$

7.3.4 F1 SCORE

F1-Score adalah metrik yang digunakan untuk mengukur keseimbangan antara presisi (precision) dan cakupan (recall) dari sebuah model klasifikasi. F1-Score merupakan rata-rata harmonik dari presisi dan cakupan, yang memberikan pandangan menyeluruh tentang kinerja model khususnya ketika kita memiliki data yang tidak seimbang (*imbalance data*). Rumus untuk menghitung F1-Score adalah:

$$F1 - Score = 2 \times \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

7.3.5 AKURASI

Akurasi adalah ukuran seberapa sering model klasifikasi membuat prediksi yang benar. Rumus yang digunakan adalah:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

7.3.6 RECEIVER OPERATING CHARACTERISTIC (ROC) CURVE

ROC Curve adalah plot dari True Positive Rate (TPR) terhadap False Positive Rate (FPR) pada berbagai *threshold*. True Positive Rate adalah Recall, sedangkan False Positive Rate didefinisikan sebagai rasio dari jumlah sampel negatif yang salah diklasifikasikan sebagai positif dibandingkan dengan total sampel negatif.

$$TPR = Recall = \frac{TP}{TP + FN}$$
$$FPR = \frac{FP}{FP + TN}$$

7.3.7 AREA UNDER THE CURVE (AUC)

AUC adalah ukuran area di bawah ROC Curve. AUC memberikan gambaran seberapa baik model dapat membedakan antara kelas positif dan negatif. Nilai AUC berkisar antara 0 dan 1, dengan 1 menunjukkan performa yang sempurna dan 0.5 menunjukkan performa yang sama dengan tebakan random (*random guessing*).

7.3.8 MATTHEWS CORRELATION COEFFICIENT (MCC)

MCC adalah ukuran korelasi antara prediksi dan observasi sebenarnya. MCC dianggap sebagai ukuran yang lebih seimbang karena mempertimbangkan semua elemen dari Confusion Matrix dan cocok

untuk *dataset* yang memiliki kelas imbang atau tidak imbang.

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

7.4 PROSEDUR PERCOBAAN

7.4.1 CONFUSION MATRIX

Untuk percobaan, dimisalkan ada label/*class* hasil prediksi (disebut *y_pred*), dan label asli dari data uji (disebut *y_true*), sebagai berikut:

```
y_true = [0, 1, 2, 1, 2, 0, 1, 0, 2, 2] # Nilai sebenarnya
y_pred = [0, 1, 1, 1, 2, 0, 2, 0, 2, 1] # Prediksi
labels = [0, 1, 2] # Label kelas
```

Variabel *labels* adalah jumlah kelas/label yang dimiliki. Dalam hal ini kelas yang digunakan berjumlah 3 kelas.

1. Percobaan Confusion Matrix

Buat *method* untuk menghitung masing-masing elemen dari confusion matrix untuk tiap kelas.

```
1 import numpy as np
2
3 def confusion_matrix_multiclass(y_true, y_pred, labels):
4     num_classes = len(labels)
5     # Inisialisasi matriks dengan ukuran (jumlah class)
6     confusion_matrix = np.zeros((num_classes, num_classes), dtype=int)
7
8     # Mengisi matriks
9     for true, pred in zip(y_true, y_pred):
10         confusion_matrix[true, pred] += 1
11
12     return confusion_matrix
13
```

Kemudian buat *method* untuk menghitung nilai TP, FP, FN, dan TN.

```
1 def calculate_metrics(confusion_matrix):
2     num_classes = confusion_matrix.shape[0]
3     metrics = {}
4
5     for i in range(num_classes):
6         TP = confusion_matrix[i, i]
7         FP = confusion_matrix[:, i].sum() - TP
8         FN = confusion_matrix[i, :].sum() - TP
9         TN = confusion_matrix.sum() - (TP + FP + FN)
10
11         metrics[labels[i]] = {
12             "TP": TP,
13             "FP": FP,
14             "FN": FN,
15             "TN": TN
16         }
17
18     return metrics
19
```

Tampilkan hasilnya jika data yang dimiliki sesuai `y_true`, `y_pred`, dan labels yang sudah didefinisikan sebelumnya.

```
1 confusion_matrix = confusion_matrix_multiclass(y_true, y_pred, labels)
2 metrics = calculate_metrics(confusion_matrix)
3
4 for label, metric in metrics.items():
5     print(f"Class {label} - TP: {metric['TP']}, FP: {metric['FP']}, FN:
      {metric['FN']}, TN: {metric['TN']}")
```

2. Percobaan Precision, Recall, F1, dan Accuracy

Menghitung Precision, Recall, F1-Score, dan Accuracy untuk tiap kelas.

```
1 for label, metric in metrics.items():
2     precision = metric['TP'] / (metric['TP'] + metric['FP'])
3     recall = metric['TP'] / (metric['TP'] + metric['FN'])
4     f1_score = 2 * (precision * recall) / (precision + recall)
5     accuracy_score = (metric['TP'] + metric['TN']) / (metric['TP'] +
      metric['FP'] + metric['FN'] + metric['TN'])
6     print(f"Class {label} - Precision: {precision}, recall: {recall}, F1-
      Score: {f1_score}, accuracy: {accuracy_score}")
```

Pada umumnya nilai F1-Score dan akurasi disajikan dalam bentuk nilai rata-rata pada setiap kelas.

7.4.2 ROC DAN AUC

Dalam kasus penghitungan ROC dan AUC ini, akan digunakan metode klasifikasi biner (hanya ada 2 kelas) sederhana dengan menggunakan metod *thresholding*. Artinya jika nilai `y_pred` $\geq$ *threshold*, maka akan diklasifikasikan ke kelas 1, dan jika `y_pred` $<$ *threshold*, maka diklasifikasikan ke dalam kelas 0. Pada kasus ini, ada banyak nilai *threshold* yang mungkin, maka dengan menggunakan kombinasi ROC dan AUC kita bisa melihat *threshold* mana yang terbaik, yaitu yang memiliki True Positive Rate (TPR) terbesar dan False Positive Rate (FPR) terkecil.

Gunakan data berikut:

```
# Contoh data
y_true = [0, 1, 1, 0, 1, 0, 1, 1, 0, 0]
y_pred = [0.1, 0.4, 0.35, 0.8, 0.7, 0.55, 0.65, 0.9, 0.3, 0.6]
```

1. Percobaan Menghitung ROC dan AUC

Buat *method* untuk model klasifikasi *thresholding*, sekaligus menghitung TPR dan FPR untuk tiap *threshold*. *Threshold* diambil dari `y_pred` yang diurutkan.

```
1 def calculate_tpr_fpr(y_true, y_pred, threshold):
2     tp = 0
3     fp = 0
4     fn = 0
5     tn = 0
6
7     for i in range(len(y_true)):
8         if y_pred[i] >= threshold:
9             if y_true[i] == 1:
10                tp += 1
11            else:
12                fp += 1
13        else:
14            if y_true[i] == 1:
```

```
15         fn += 1
16     else:
17         tn += 1
18
19     tpr = tp / (tp + fn)
20     fpr = fp / (fp + tn)
21
22     return tpr, fpr
23
```

Lalu buat *method* untuk mengitung nilai ROC dan AUC.

```
1 def calculate_roc_auc(y_true, y_pred):
2     thresholds = sorted(set(y_pred), reverse=True)
3     roc_points = []
4
5     for threshold in thresholds:
6         tpr, fpr = calculate_tpr_fpr(y_true, y_pred, threshold)
7         roc_points.append((fpr, tpr))
8
9     # Menghitung AUC menggunakan metode trapezoid
10    auc = 0.0
11    for i in range(1, len(roc_points)):
12        x1, y1 = roc_points[i - 1]
13        x2, y2 = roc_points[i]
14        auc += (x2 - x1) * (y1 + y2) / 2.0
15
16    return roc_points, auc
17
18
```

Panggil *method* tersebut, dan plot hasilnya dengan menggunakan **pyplot**.

```
1 import matplotlib.pyplot as plt
2
3 # Menghitung ROC dan AUC
4 roc_points, auc = calculate_roc_auc(y_true, y_pred)
5
6 print("ROC Points:", roc_points)
7 print("AUC:", auc)
8
9 # Plot ROC curve
10 x, y = zip(*roc_points)
11 plt.plot(x, y, marker='o')
12 plt.xlabel('False Positive Rate')
13 plt.ylabel('True Positive Rate')
14 plt.title('ROC Curve')
15
16 # Show grid
17 plt.grid(True)
18
19 # Show plot
20 plt.show()
21
```

7.5 TUGAS

1. Buat *method* untuk menghitung F1-Score dan accuracy rata-rata
2. Buat kode program untuk menghitung MCC dari percobaan pada Subbab 7.4
3. Gunakan semua matriks evaluasi yang ada di bab ini untuk menghitung kinerja SVM pada *dataset* iris, yang sudah dipraktikkan pada bab sebelumnya.

7.6 KESIMPULAN

Modul praktikum ini telah berhasil mengimplementasikan dan mengevaluasi berbagai metode klasifikasi menggunakan matriks evaluasi dalam bahasa Python. Hasil yang diperoleh menunjukkan bahwa matriks evaluasi seperti akurasi, presisi, recall, dan F1-score sangat penting dalam menilai kinerja model klasifikasi. Penggunaan matriks evaluasi ini membantu dalam memilih model yang paling efektif dan andal untuk diterapkan pada data yang relevan.

Setelah melakukan semua percobaan ini maka mahasiswa diwajibkan membuat kesimpulan.

Kesimpulan dari percobaan ini merupakan bagian penting untuk merangkum temuan utama dan relevansinya dengan tujuan percobaan. Mahasiswa harus mampu menyimpulkan:

- Apakah tujuan percobaan telah tercapai?
- Hubungan antara hasil percobaan dan dasar teori.
- Faktor-faktor yang memengaruhi hasil percobaan, termasuk kemungkinan kesalahan yang terjadi.

Kesimpulan harus disusun secara singkat, jelas, dan berdasarkan data yang diperoleh.